

Looking for All Palindromes in a String

Shih Jang Pan and R. C. T. Lee

Department of Computer Science and Information Engineering, National Chi-Nan

University, Puli, Nantou Hsien, 545, Taiwan, ROC

sjpan@algdod.csie.ncnu.edu.tw, rctlee@ncnu.edu.tw

Abstract

A palindrome is a string of the form aa' , where a and a' are also strings and reverse to each other. The problem of the paper is defined as follows: given a string S of length n , find all palindromes occurring in the given string S . In the paper, we present an algorithm based on suffix trees to find palindromes.

Our algorithm will find all maximum palindromes which are not contained in any other palindromes. After finding maximum palindromes, we utilize eliminating operations to find other palindromes which are contained in maximum palindromes.

1 Introduction

Let us first define palindromes. Given a string $\alpha = \alpha_1\alpha_2\cdots\alpha_n$, $\alpha' = \alpha_n\alpha_{n-1}\cdots\alpha_1$ is called the reverse of α . The string $\alpha\alpha'$ is called a palindrome. For example, *abba* and *cdaadc* are both palindromes. To look for DNA sequences containing palindromes is important in biology. Many researchers have discussed the palindrome problem [2] [3] [4].

Suffix trees have been used extensively by many algorithms [1]. Gusfield proposed an algorithm based on suffix trees to find all tandem repeats in a given DNA sequence [5]. In this paper, we modify the idea in [5] and present a suffix tree approach to deal with the palindrome problem.

2 A suffix tree approach to detect all palindromes

2.1 Term definition

Given an input string K , we first add a character $\$$ which does not occur in K to the end of the input string. The new string is denoted as K_1 . Then, we reverse the given string K and add a character $\$$ which does not occur in the string to the end of the reversed string. The new string is denoted as $-K_1$. We now construct a suffix tree for both K_1 and $-K_1$.

Assume that we have $K_1 = cbaab\$$ and $-K_1 = baabc\$$. We will construct a suffix tree for

K_1 and $-K_1$. To clearly describe the suffix tree, we need many indices for nodes of it. They will be explained below.

- (1) Rule 1: Each leaf node is associated with either K_1 or $-K_1$, indicated by the $\$$ or $\$'$ on the edge ending at the leaf node as indicated in Fig. 1(a).
- (2) Rule 2: If a leaf node is associated with $K_1(-K_1)$ there is an index $i_f(i_r)$, indicating the starting location of the associated string. The index $i_f(i_r)$ is in regular type (italic type) as indicated in Fig. 1(a).
- (3) Rule 3: For each node, except the root node, the collection of leaf indices below the node is called forward collection or reversed collection for K_1 or $-K_1$. The indices collected in forward and reversed collections are enclosed in () parentheses. If the numbers inside the parenthesis are in regular type (italic type), they are associated with $K_1(-K_1)$ as indicated in Fig. 1(b).

We further have Rule 4 as follows:

- (4) Rule 4: For each node, the length from the root to the node is denoted as $D(v)$. $D(v)$ is shown in [] bracket as shown in Fig. 2.

The above results are merged and shown in Fig. 3. For our algorithm of palindrome finding, we need the following terms as defined below.

- (1) Eliminating operation: Given a palindrome string $S = s_1s_2\cdots s_n$, the eliminating operation deletes the first and last characters of the given string S . The remaining string $S' = s_2\cdots s_{n-1}$ is also a palindrome. For example, *abba* is generated from *cabbac* by an eliminating operation.
- (2) Maximum Palindromes: A substring T of S is a maximum palindrome of S if T is not contained any other palindrome of S . For example, in the string *dcabbace*, the substring *cabbac* is a maximum palindrome while *abba* is not.

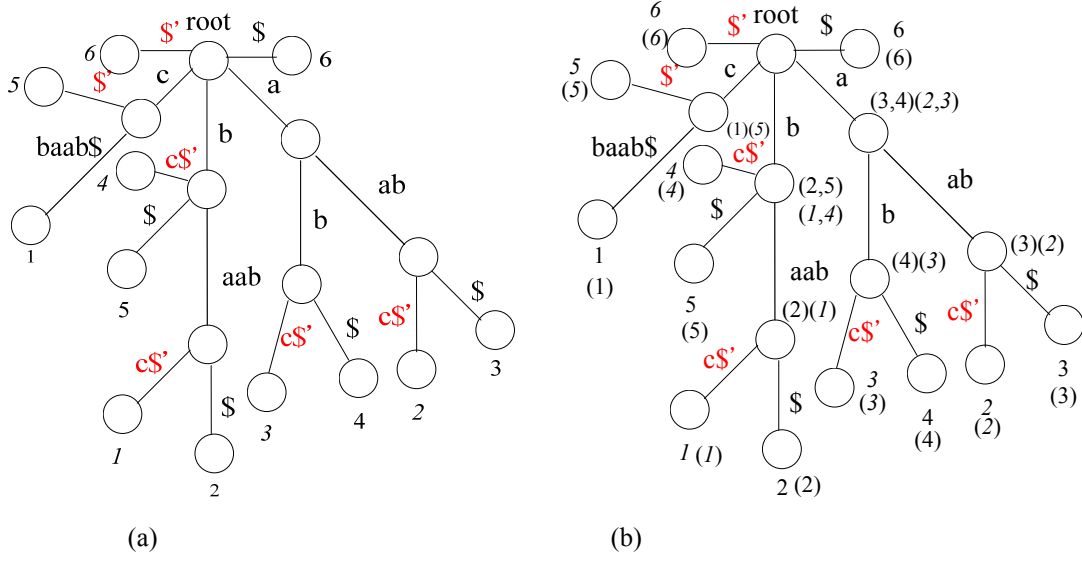
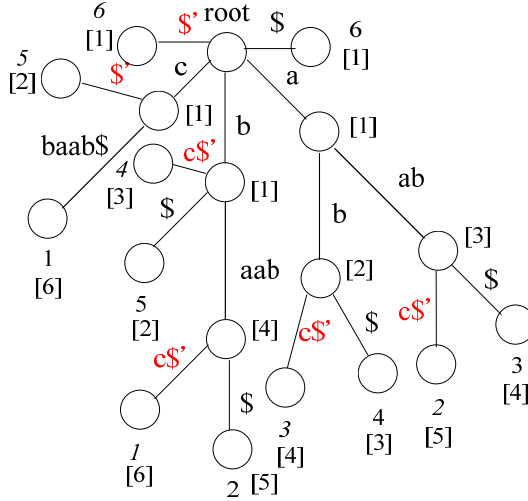

 Figure 1 $K_1 = cbaab\$$ and $-K_1 = baabc\$'$


Figure 2

2.2 The Method

In this section, we first present the algorithm based on suffix trees to find all maximum palindromes in a given string. We construct the suffix tree of K_1 and $-K_1$. At each node v , we check whether there exist i_f and i_r such that $i_f + i_r = (n - D(v) + 2)$. If the substring associated with this node is a maximum palindrome, we will find i_f and i_r such that $i_f + i_r = (n - D(v) + 2)$. On the other hand, if $i_f + i_r = (n - D(v) + 2)$ for some i_f and i_r , the substring corresponding to this node must be a palindrome, but not necessarily a maximum palindrome.

We now use the input string $cbaab$ to explain the above discussion. Consider Fig. 4 whose suffix tree is constructed for $cbaab\$$ and $baabc\$'$. We are

interested in the node indicated by the arrow sign.

For this pointed node, $i_f = 2$, $i_r = 1$, $n = 5$, and $D(v) = 4$. Thus $i_f + i_r = (n - D(v) + 2)$.

Thus $i_f + i_r = (n - D(v) + 2)$. The substring corresponding to this node is $baab$ and is a maximum palindrome. By using an eliminating operation, we get another palindrome aa . For other nodes of the suffix tree, the formula does not hold. So there are not other maximum palindromes.

We now show another example $cabbaabb$. The suffix tree is shown in Fig. 5.

For the left pointed node, $n = 8$ and $D(v) = 4$. Besides there exist $i_f = 2$, and $i_r = 4$. Thus the formula

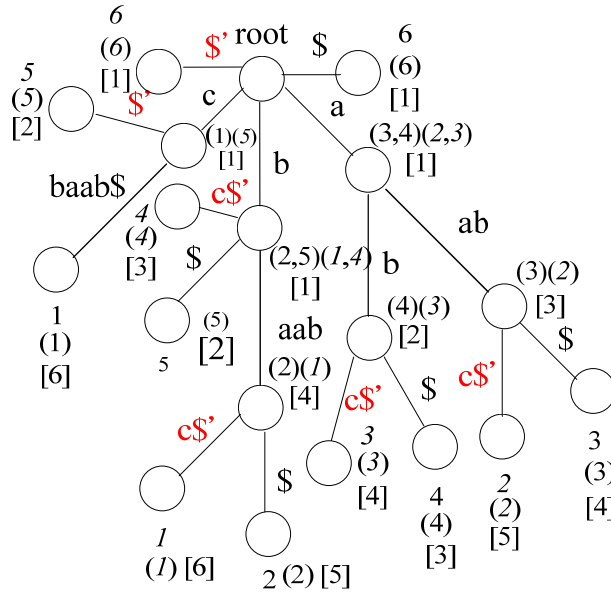


Figure 3

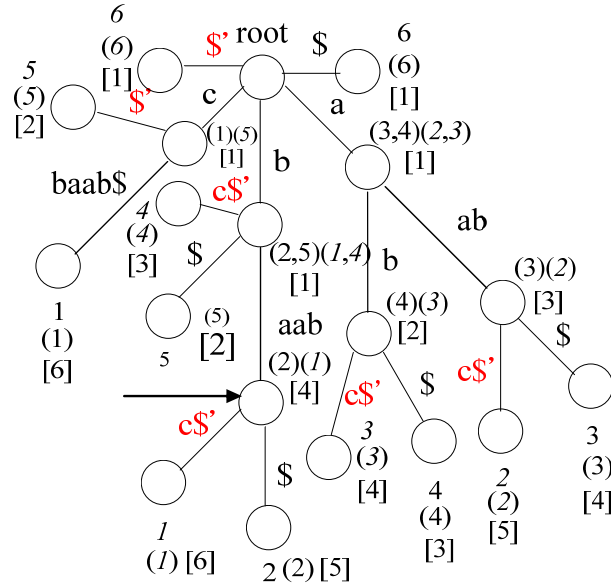


Figure 4

$i_f + i_r = (n - D(v) + 2)$ holds for such i_f and i_r . The substring corresponding to this node is *abba* which is a maximum palindrome.

By using eliminating operation, we get another palindrome *bb*. For other nodes, the formula does not hold. So there are not other maximum palindromes. We can easily see that we can find another two maximum palindromes at the other two pointed nodes. They are *bbaabb* and *bb*. We would like to point out that we may find another *bb* which is a palindrome, but not a maximum palindrome.

In the following, we present the algorithm for

finding all maximum palindromes.

Algorithm

Input: a string S with length n .

Output: all palindromes which occur in S .

1. Add a character $\$$ which does not occur in S to the end of S . The new string is denoted as S' .
2. Add a character $\$'$ which does not occur in the string to the end of the reversed input string $-S$. The new string is denoted as $-S'$.
3. Construct the suffix tree for S' and $-S'$.

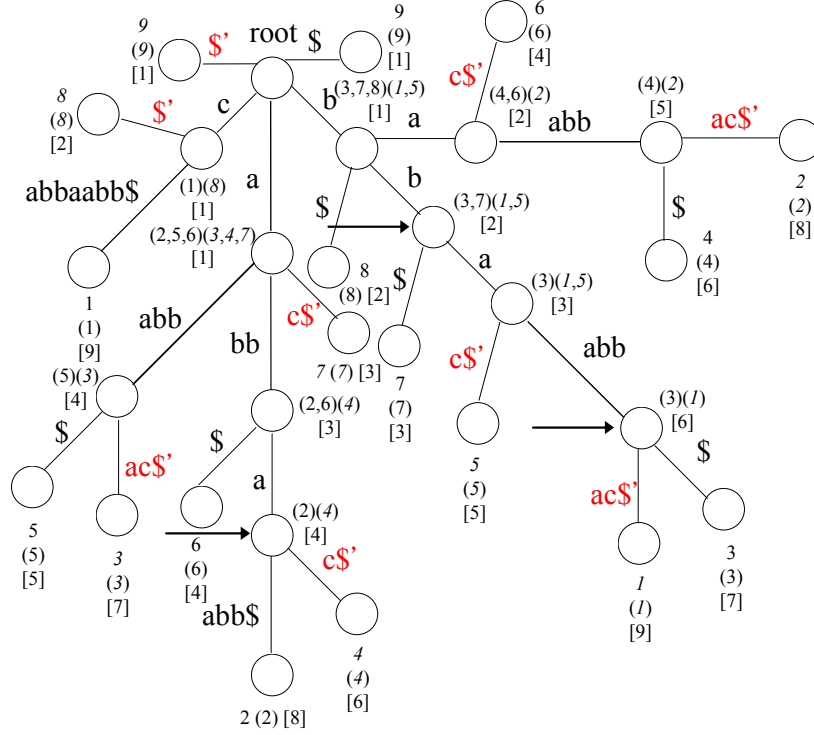


Figure 5

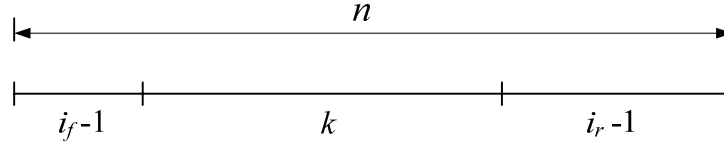


Figure 6

4. Collect the forward, reversed collections for S' or $-S'$ and find $D(v)$ for each node in the suffix tree.
5. For each node v of the suffix tree, check whether there exist i_f and i_r such that $i_f + i_r = (n - D(v) + 2)$. If yes, return the substring corresponding to this node v as a possible maximum palindrome of S .
6. Utilize eliminating operations to all palindromes obtained in the above procedures to produce more palindromes

3 The correctness of our proposed algorithm

We finally discuss why the algorithm works. Let us assume that there is a maximum palindrome starting at i_f with length k in the given string S . Then there must be a maximum palindrome starting at i_r in $-S$. As shown in Fig. 6, we have

$$i_f - 1 + k + i_r - 1 = n$$

But, on the suffix tree constructed out of S and $-S$, we have:

$$K = D(v)$$

Thus, we have $i_f + i_r = (n - D(v) + 2)$.

4 Conclusions

In this paper, we presented an algorithm based on suffix trees to find all palindromes in a given string. The main goal is actually to define a new kind of palindromes, namely palindromes with gaps between α and α' .

We have already some preliminary result to indicate that our present algorithm, with modification, can be used to find the new kind of palindromes, which cannot be found by using the Manacher algorithm [2]. This is our future research.

Reference

- [1] D. Gusfield. Algorithms on Strings, Trees, and Sequences. Cambridge University Press, 1997.
- [2] G. Manacher. A New linear-Time "On-Line" Algorithm for Finding the Smallest Initial Palindrome of a String. *Journal of the Association for Computing Machinery*, 1975.
- [3] A. Porto, V. Barbosa. Finding Approximate Palindromes in Strings. *Pattern Recognition*, 2002.
- [4] R. Gupta, A. Mittal, V. Narang, S. Wing-Kin. Detection of Palindromes in DNA Sequences Using Periodicity Transform. IEEE International Workshop on Biomedical Circuits & Systems, 2004.
- [5] J. Stoye, D. Gusfield. Simple and flexible detection of contiguous repeats using a suffix tree. *Theoretical Computer Science*, 2002.